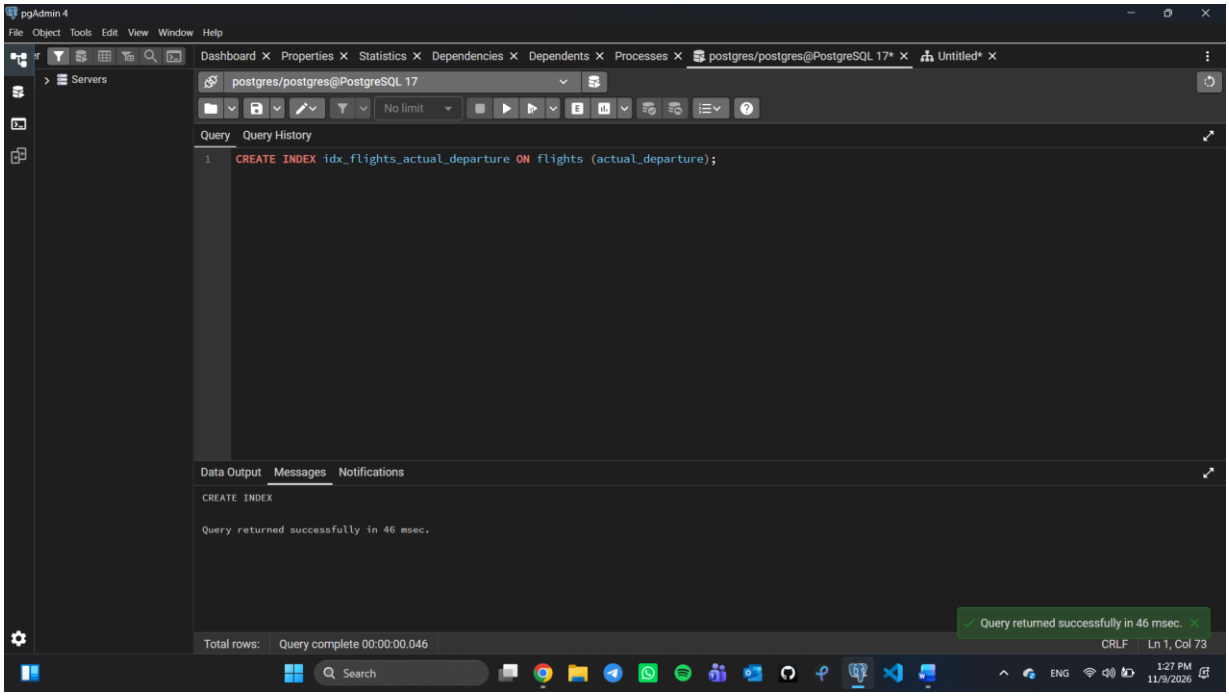
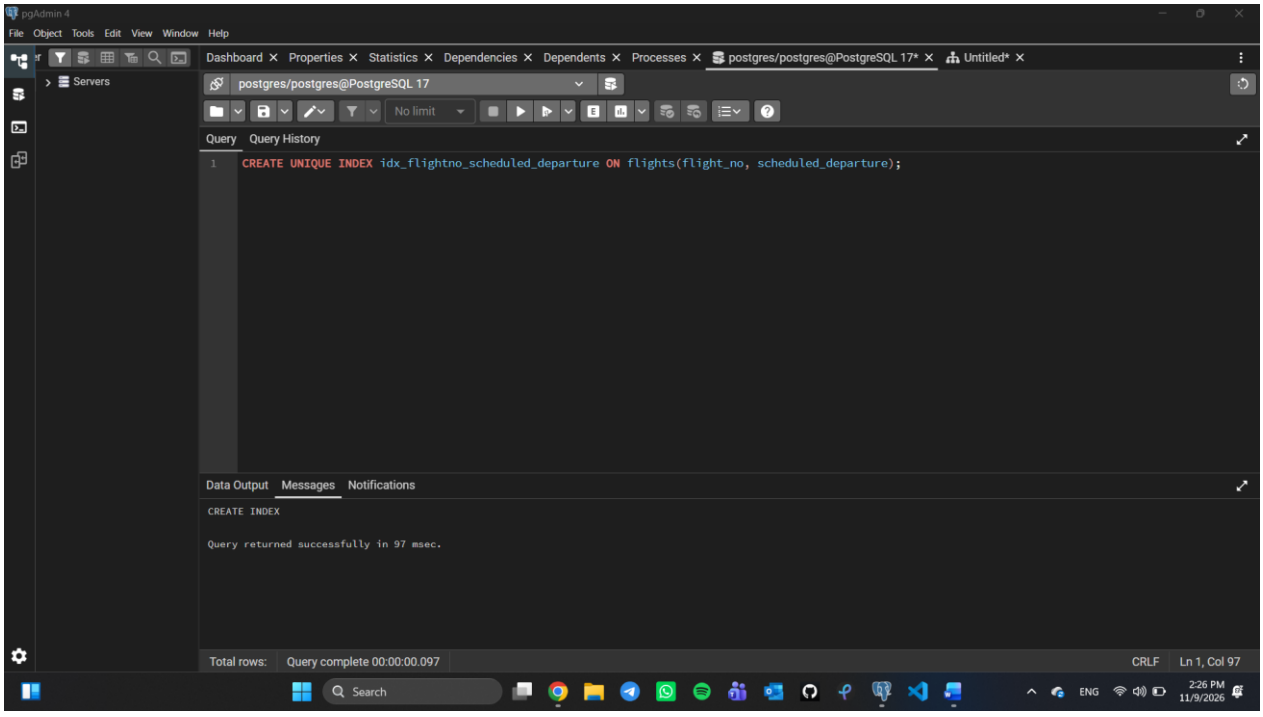
Laboratory work 7

INDEX

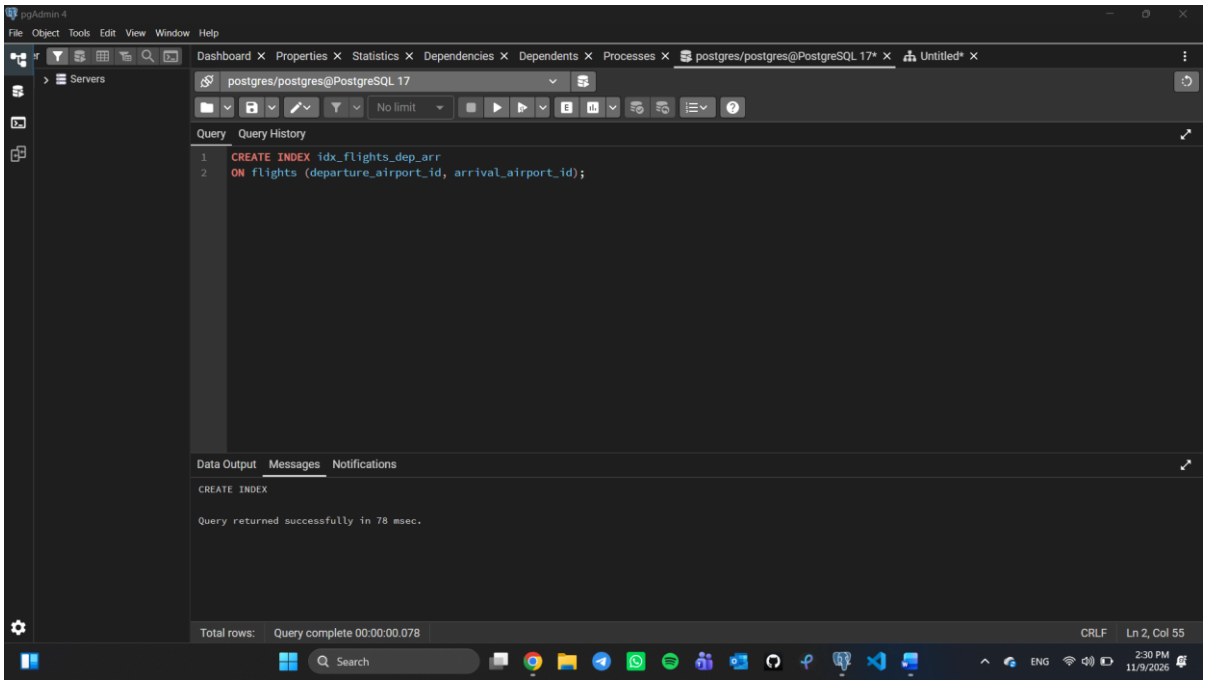
1. Create an index on the actual_departure column in the flights table.



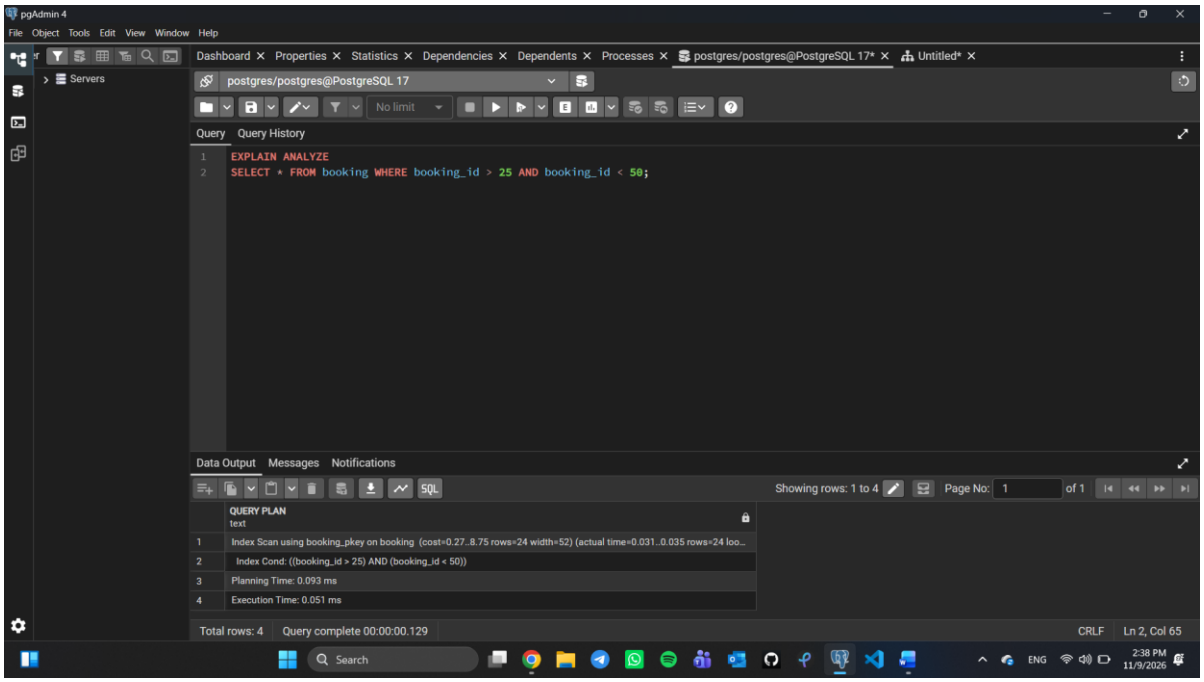
2. Create a unique index to ensure flight_no and scheduled_departure combinations are unique.

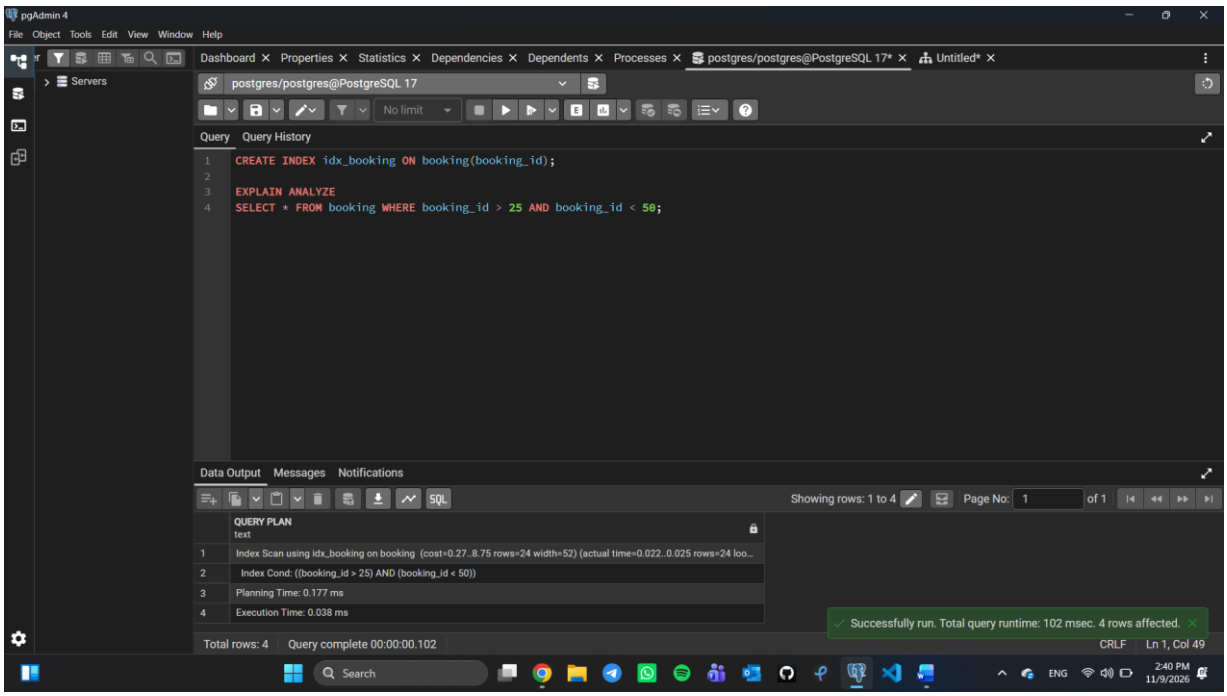


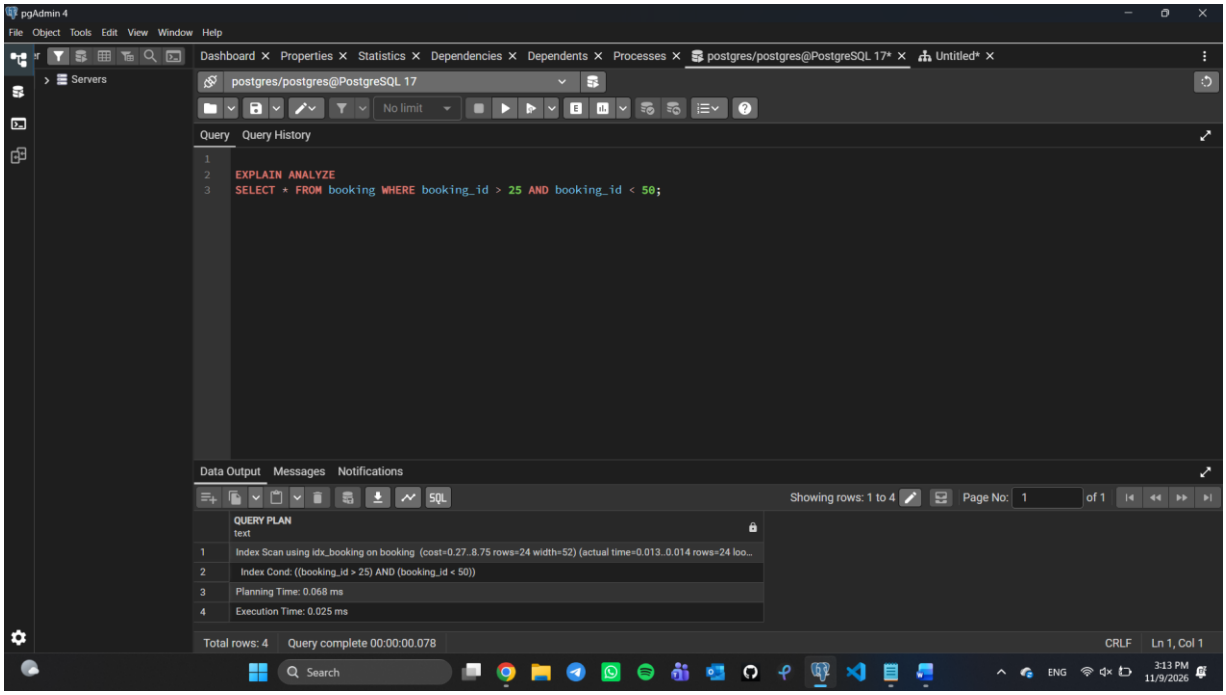
3. Create a composite index on the departure_airport_id and arrival_airport_id columns.



4. Evaluate the difference in query performance with and without indexes. Measure performance differences.



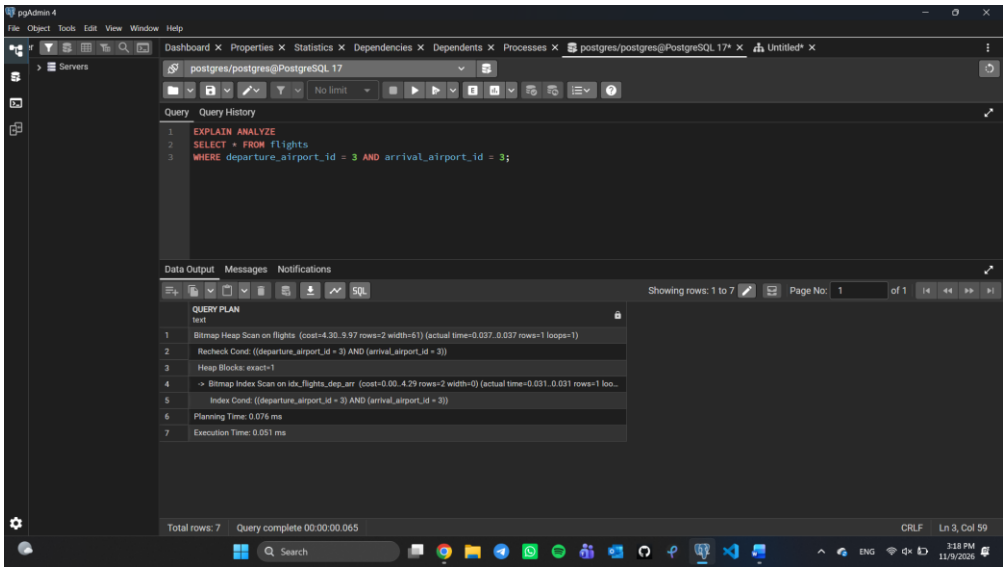




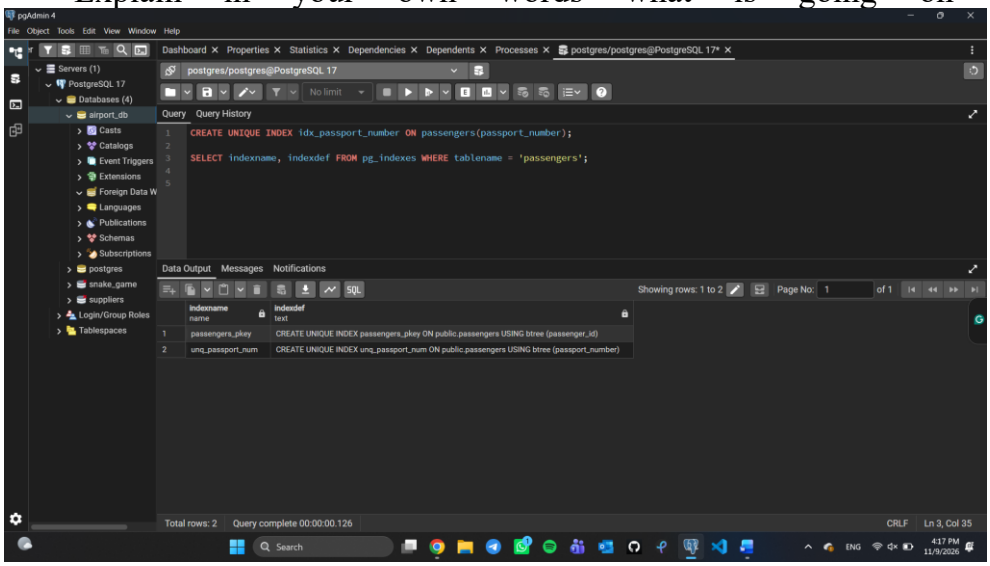
Comparing two runs with and without index, we can see that the index lets us run query slightly faster.

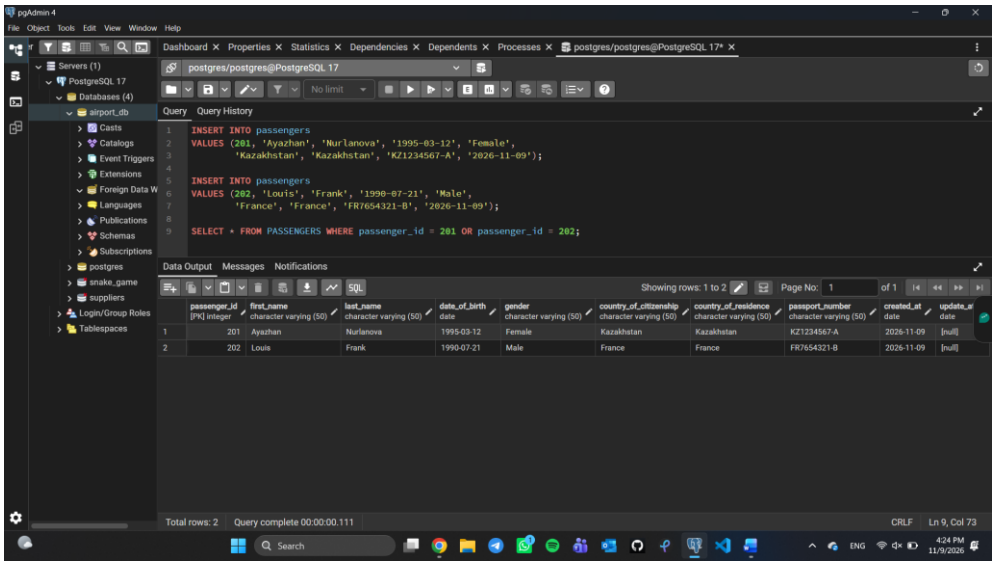
Indexes help most on large tables with selective filters, while on small tables the gain is minimal and the planner may skip the index.

5. Use EXPLAIN ANALYZE to check index usage in a query filtering by departure_airport and arrival_airport.



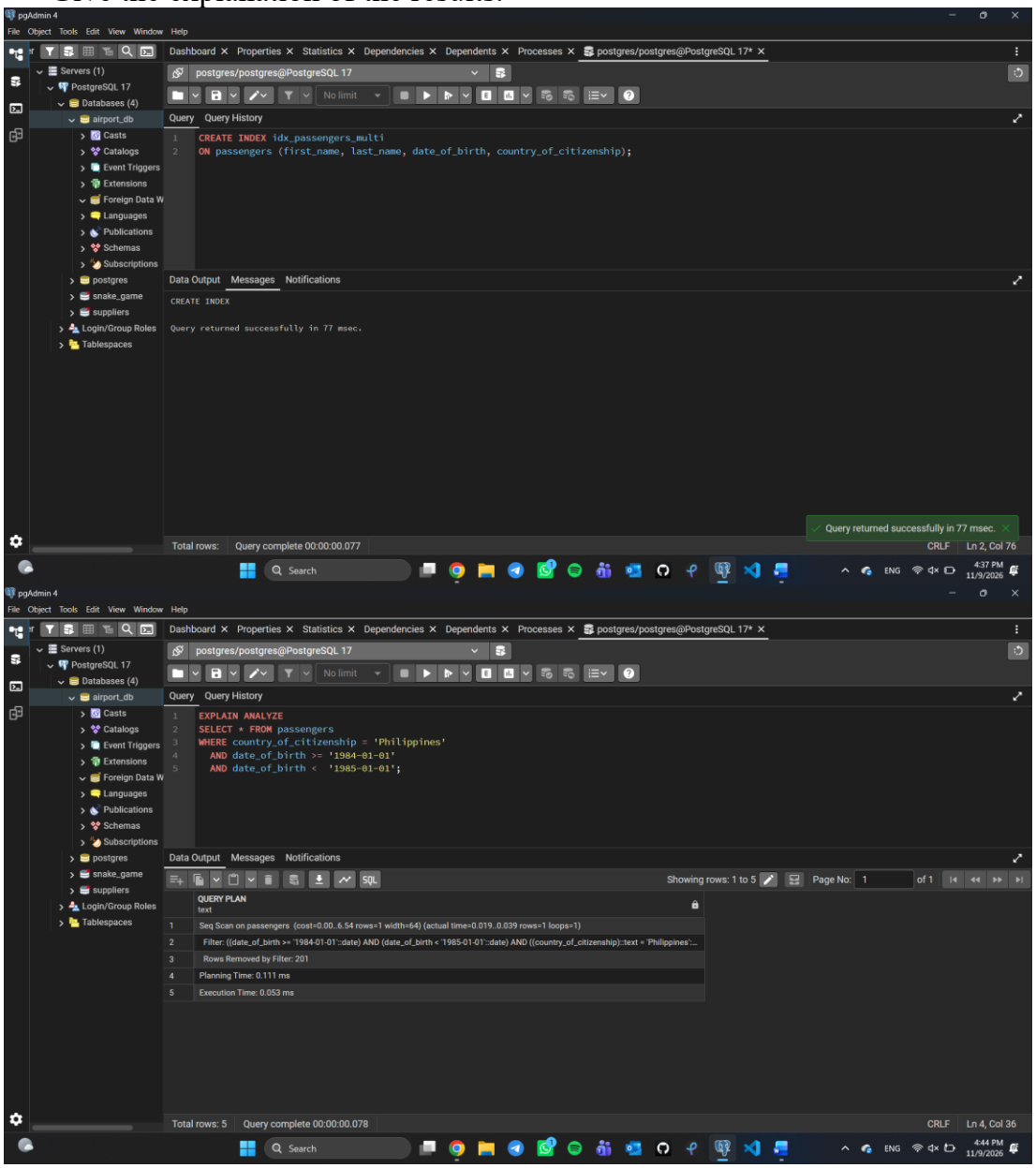
6. Create a unique index for the passport_number of the Passengers table. Check if the index was created or not. Insert into the table two new passengers.
Explain in your own words what is going on in the output?





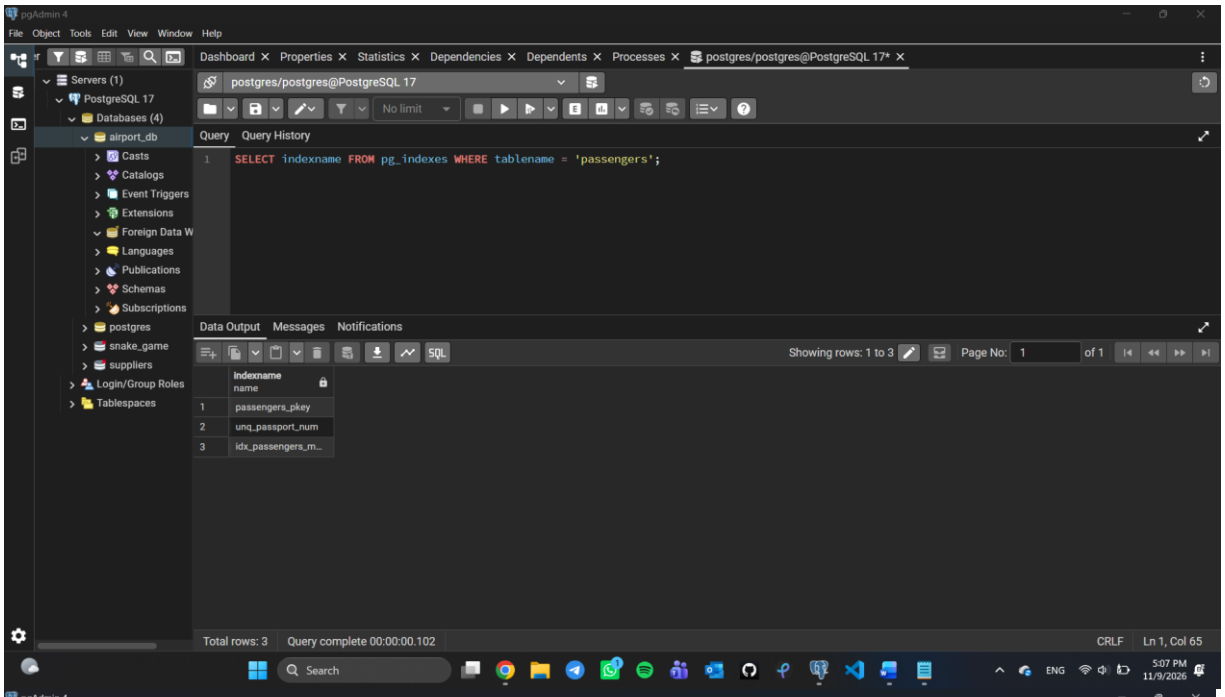
I created a unique index on passport_number and checked it in pg_indexes, where it appears as a B-tree index with the unique option. Then I inserted two passengers with different passport numbers, and both were accepted. The index speeds up searching by passport number, but also guarantees that every passport number in the table is unique.

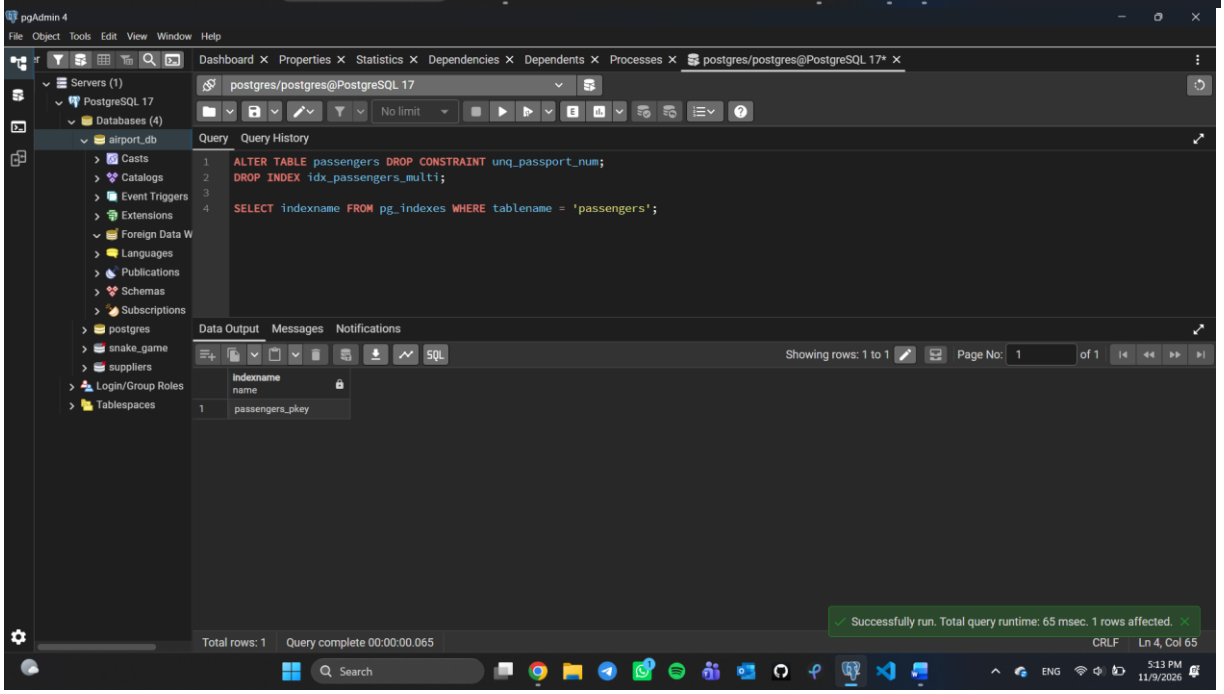
7. Create an index for the Passengers table. Use for that first name, last name, date of birth and country of citizenship. Then, write a SQL query to find a passenger who was born in Philippines and was born in 1984 and check if the query uses indexes or not. Give the explanation of the results.



Query filters only on the third and fourth columns, and does not mention first_name or last_name. The index is sorted by the first column first, so a query that skips the leading columns can not use it efficiently.

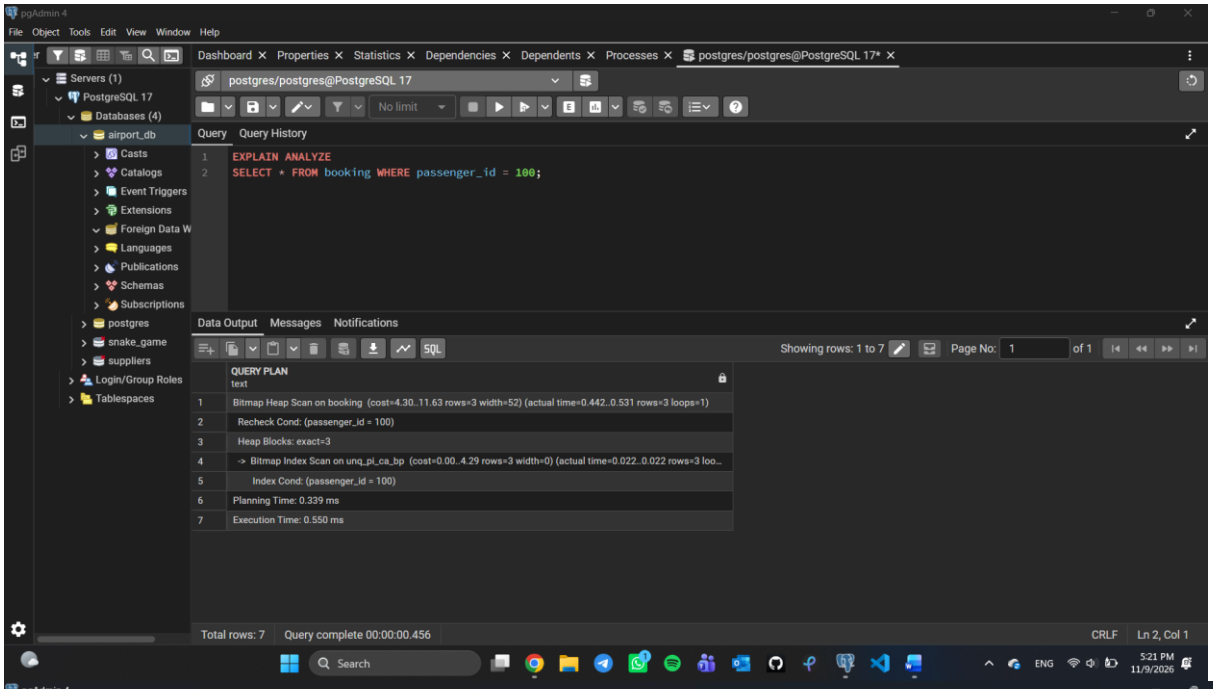
8. Write a SQL query to list indexes for table Passengers. After delete the created indexes.

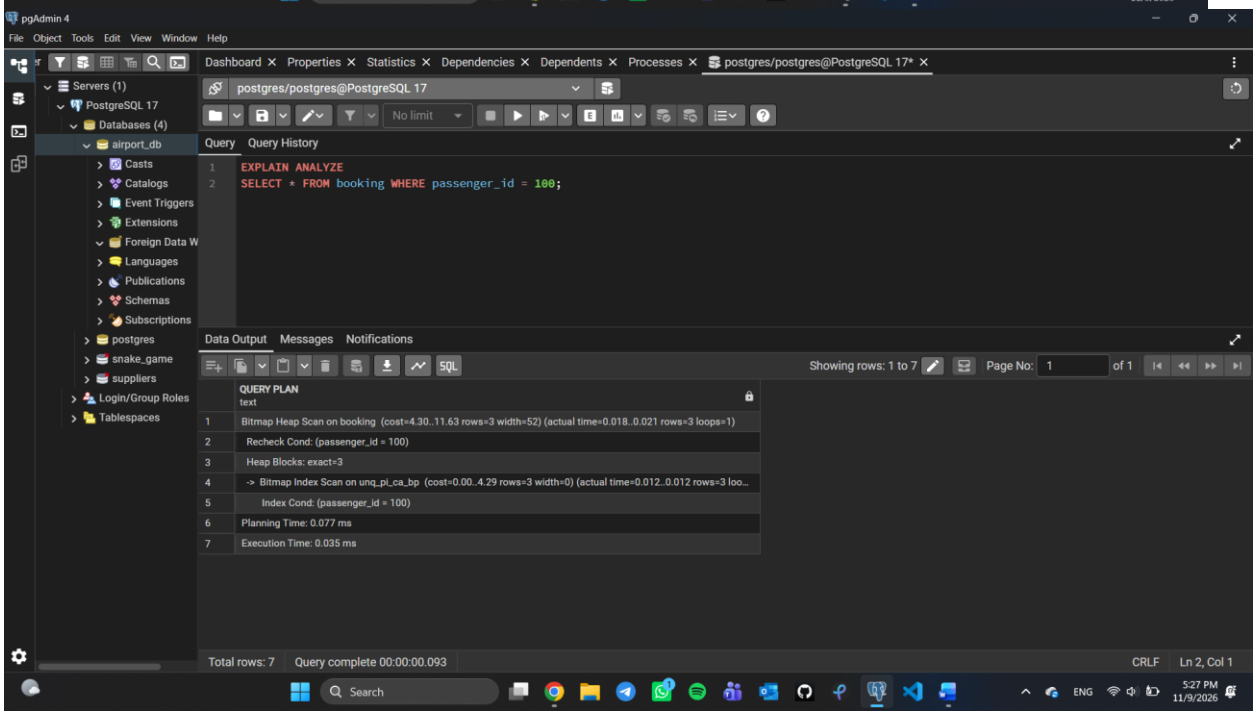




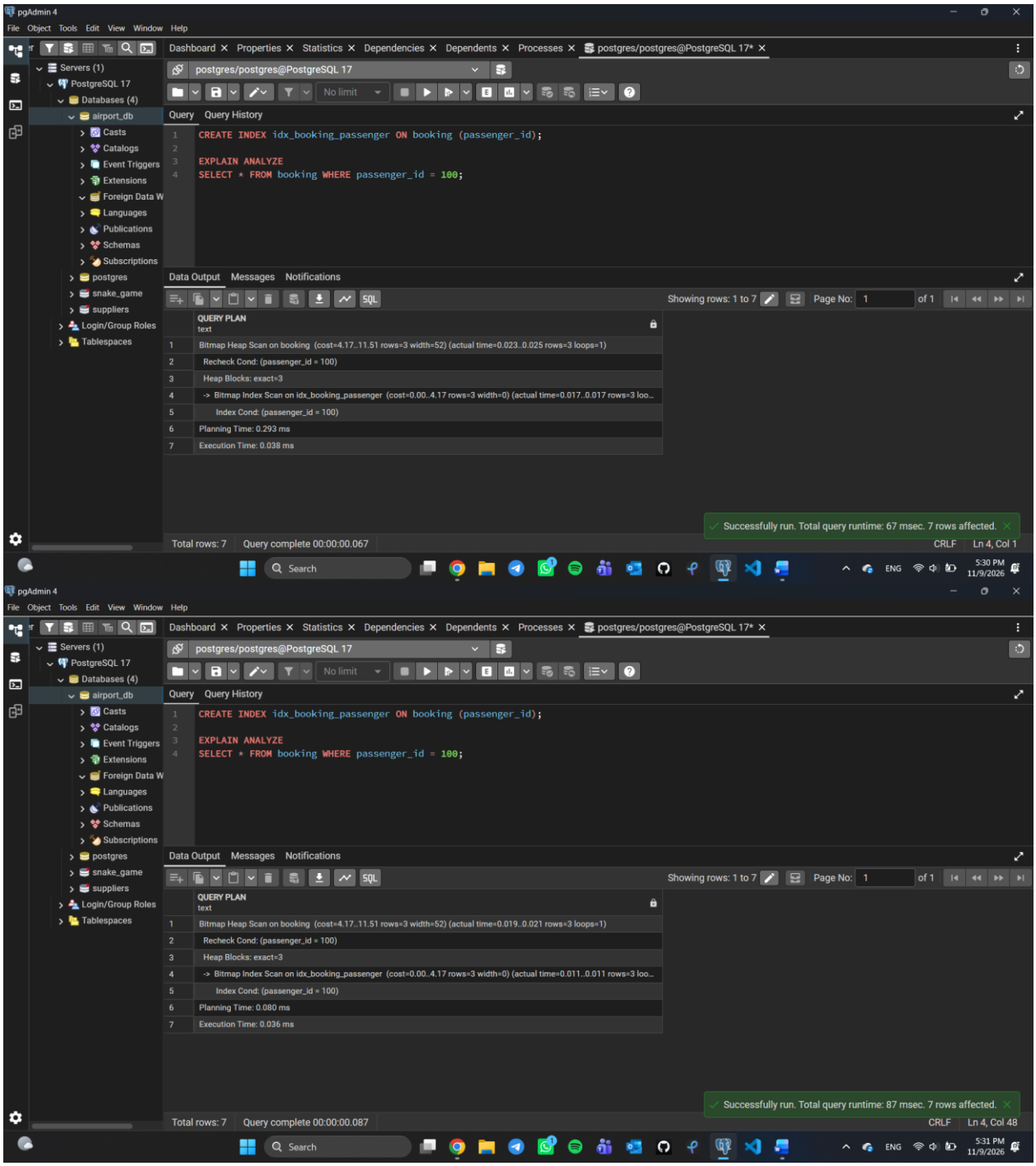
9. Create an index on the passenger_id column of the booking table. Using EXPLAIN ANALYZE, compare the execution plan and execution time of a query that selects all bookings of one passenger (for example, passenger_id = 100) before and after creating the index. Explain what changed in the plan.

Before



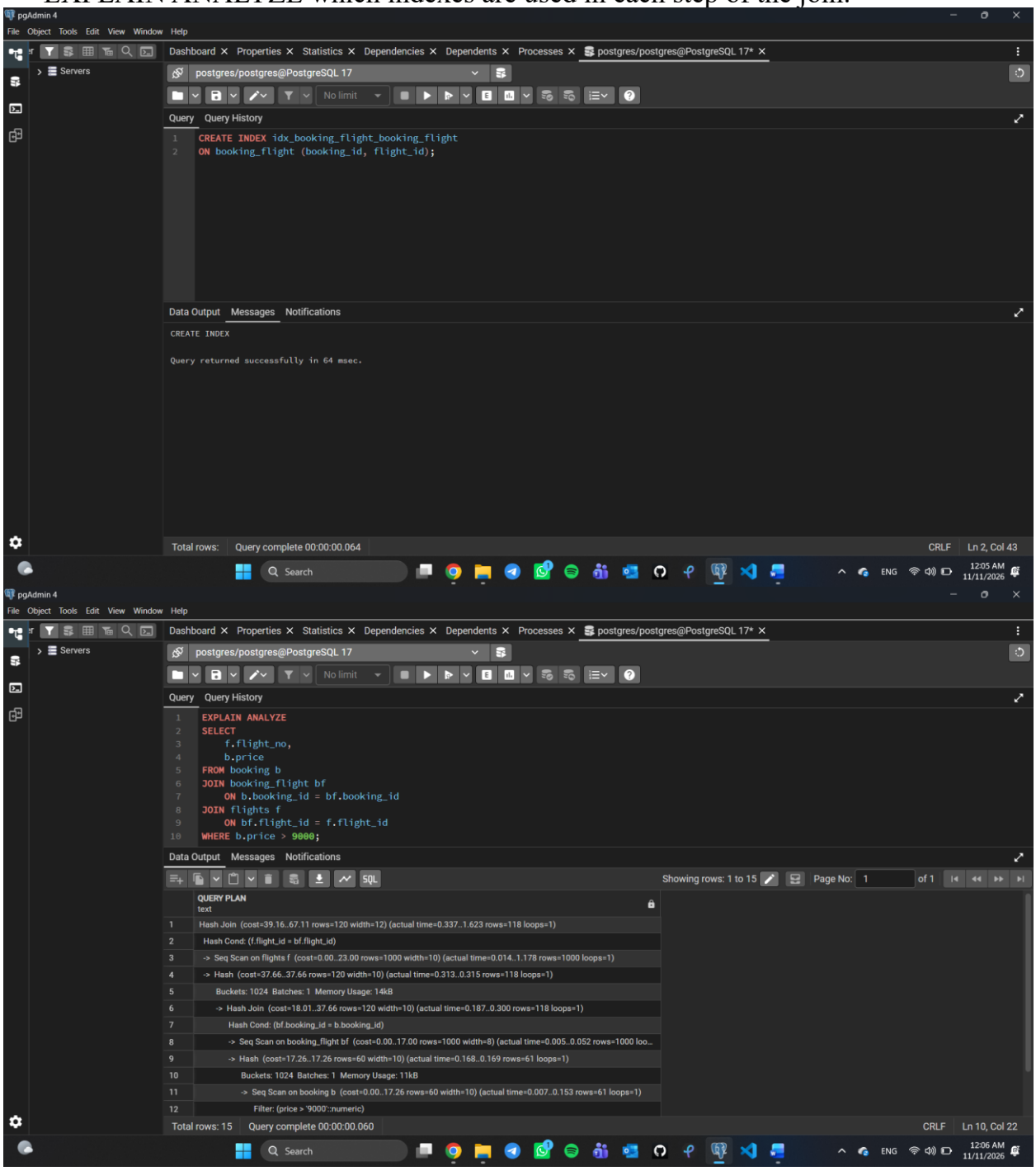


After

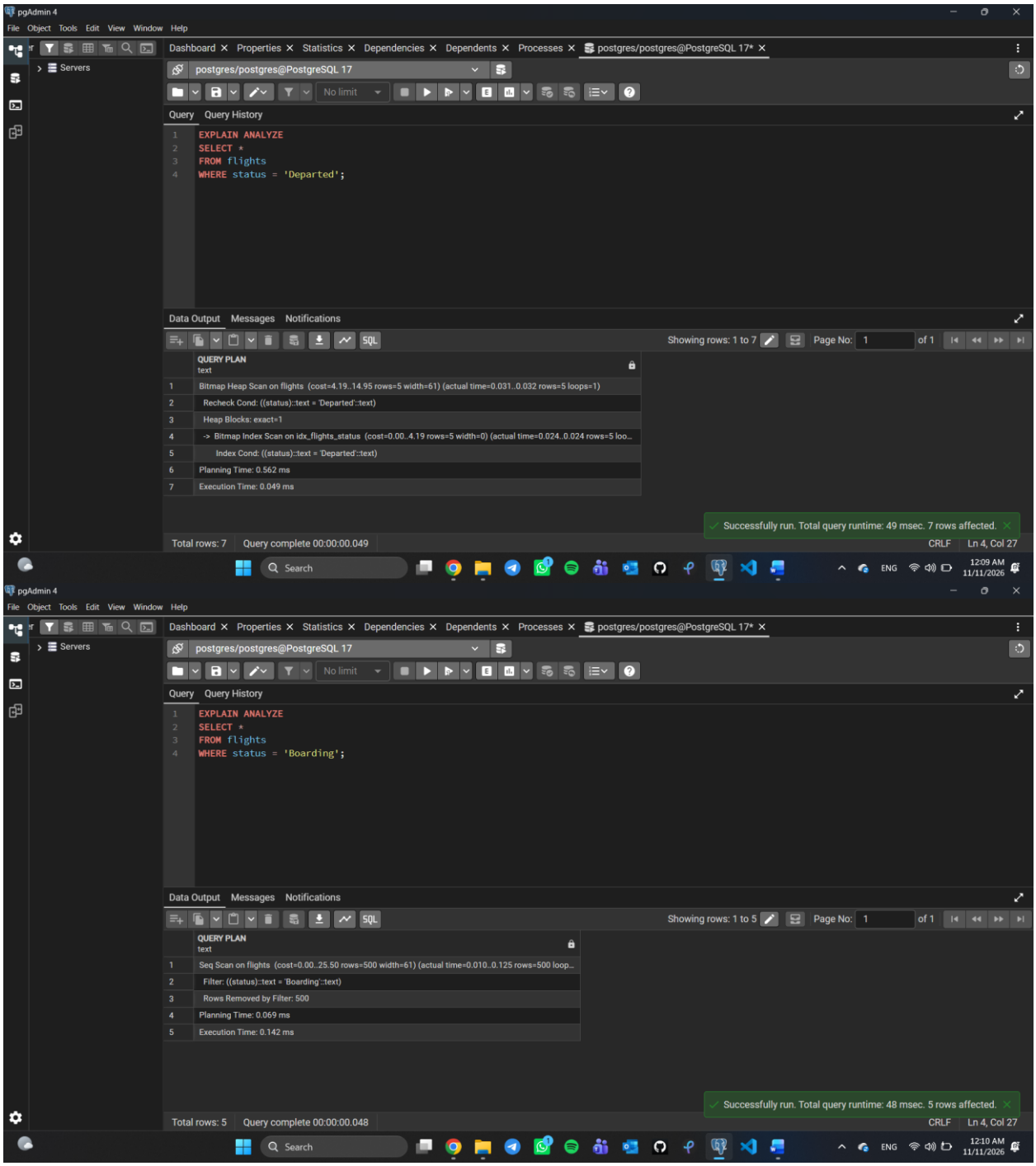


The booking table is small, and an existing index already covered this query, so the new index did not make it noticeably faster. The first run took 0.550 ms only because the data was not yet cached. On a large table with no suitable index, the same query would use a Seq Scan and read every row, and an index on passenger_id would let PostgreSQL go straight to the matching rows.

10. Create a composite index on the booking_flight table using the booking_id and flight_id columns. Write a query that joins booking, booking_flight and flights and returns the flight_no and price for bookings with a price greater than 9000. Check with EXPLAIN ANALYZE which indexes are used in each step of the join.



11. Create an index on the status column of the flights table. Run EXPLAIN ANALYZE for the query with status = 'Departed' and for the query with status = 'Boarding'. Compare the plans and explain, in your own words, why the optimizer may ignore the index for one of the values (hint: selectivity).

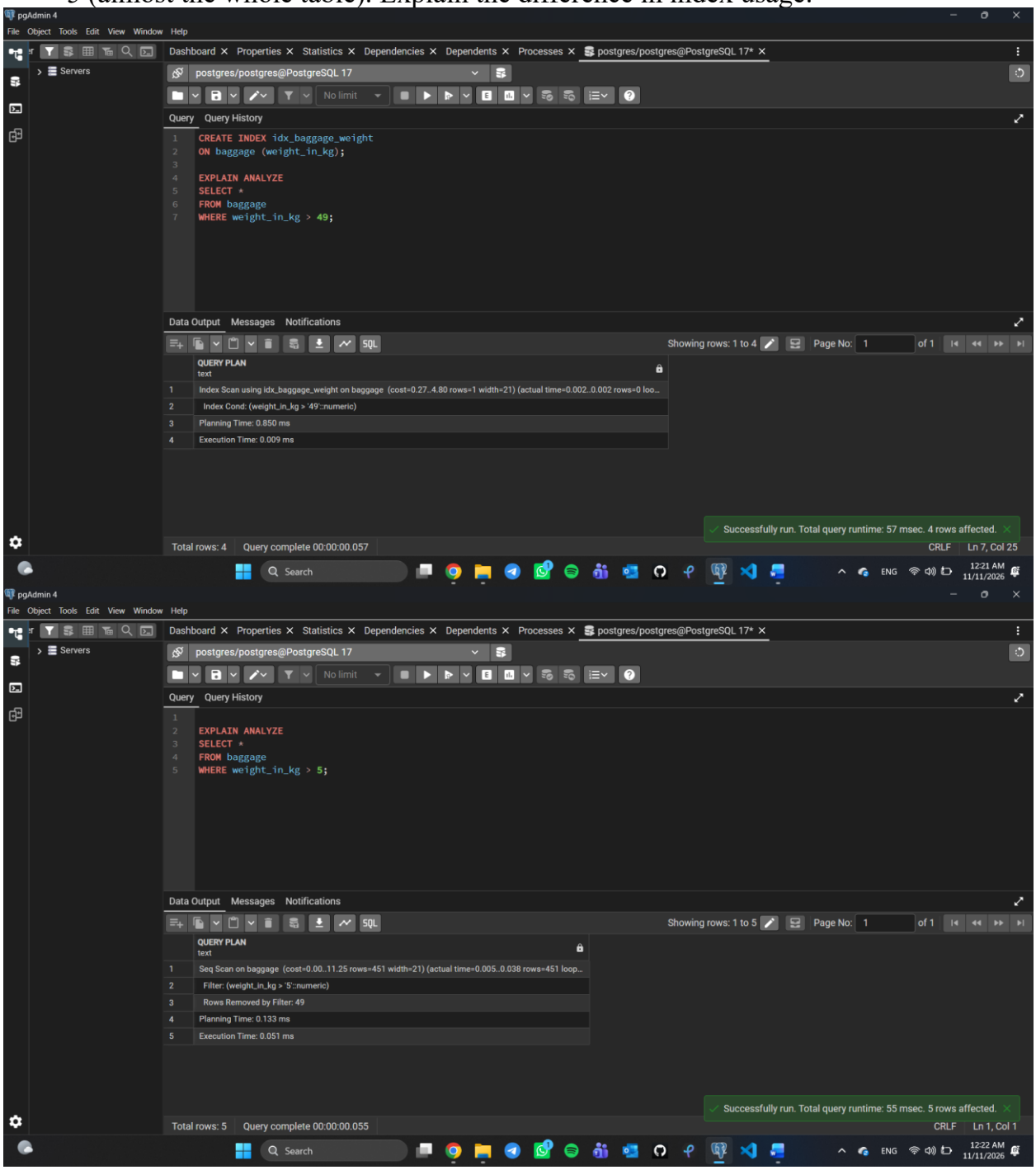


For the query with status 'Departed' used a Bitmap Index Scan followed by a Bitmap Heap Scan. It found 5 matching rows.

For the query with status 'Boarding'index. It also found 5 matching rows, but the optimizer estimated a much larger number of matching rows.

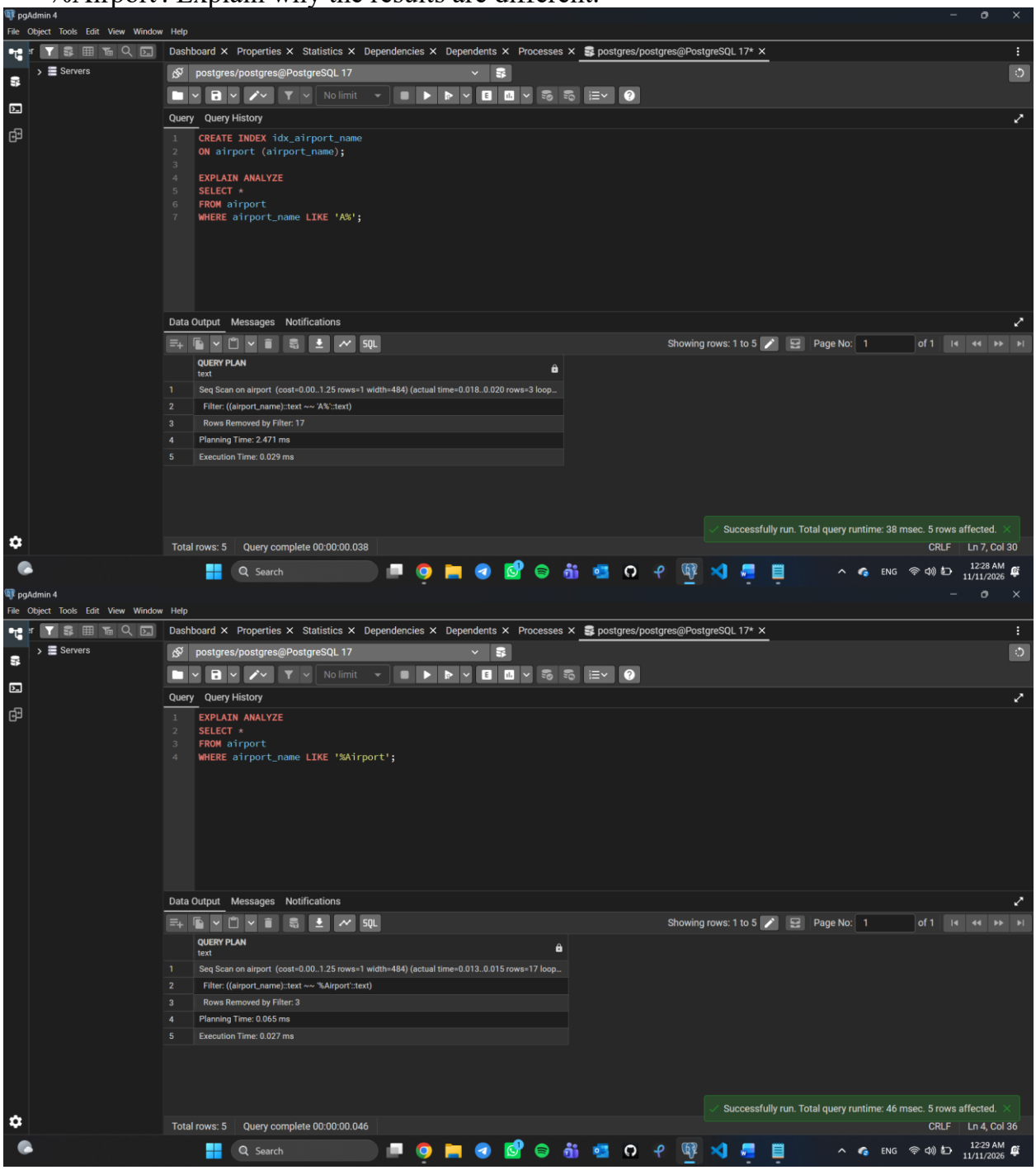
Optimizer may ignore the index for a value that matches many rows because reading the entire table can be more efficient than accessing many rows through the index.

12. Create an index on the weight_in_kg column of the baggage table. Compare the plans of two range queries: weight_in_kg > 49 (a small part of the table) and weight_in_kg > 5 (almost the whole table). Explain the difference in index usage.



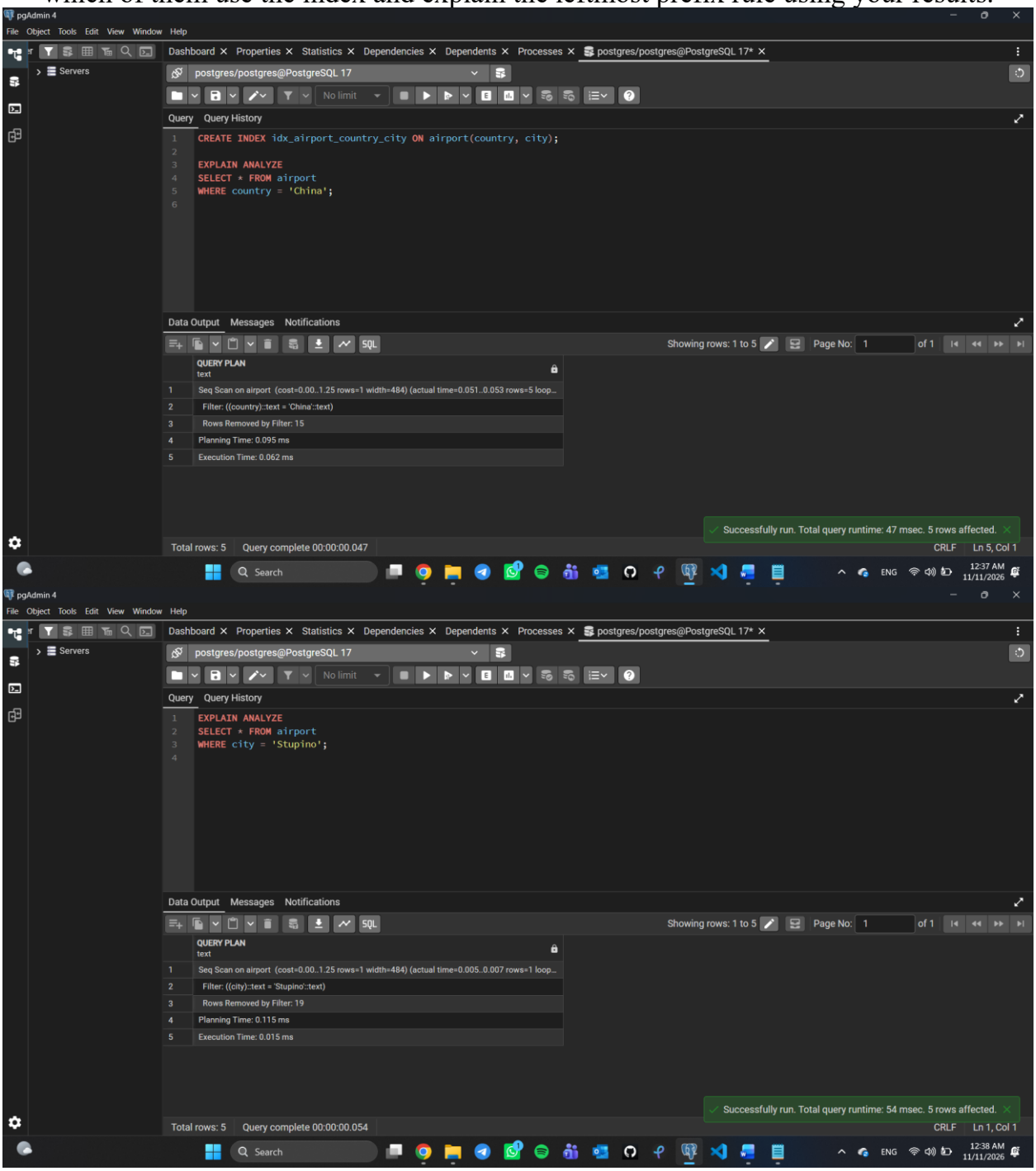
For the first query retrieved only 4 rows. Since this condition matches a very small part of the table, using the index is efficient For the second query 451 rows. Since almost the entire table satisfies this condition, scanning the whole table is more efficient. This shows that index usage depends on selectivity.

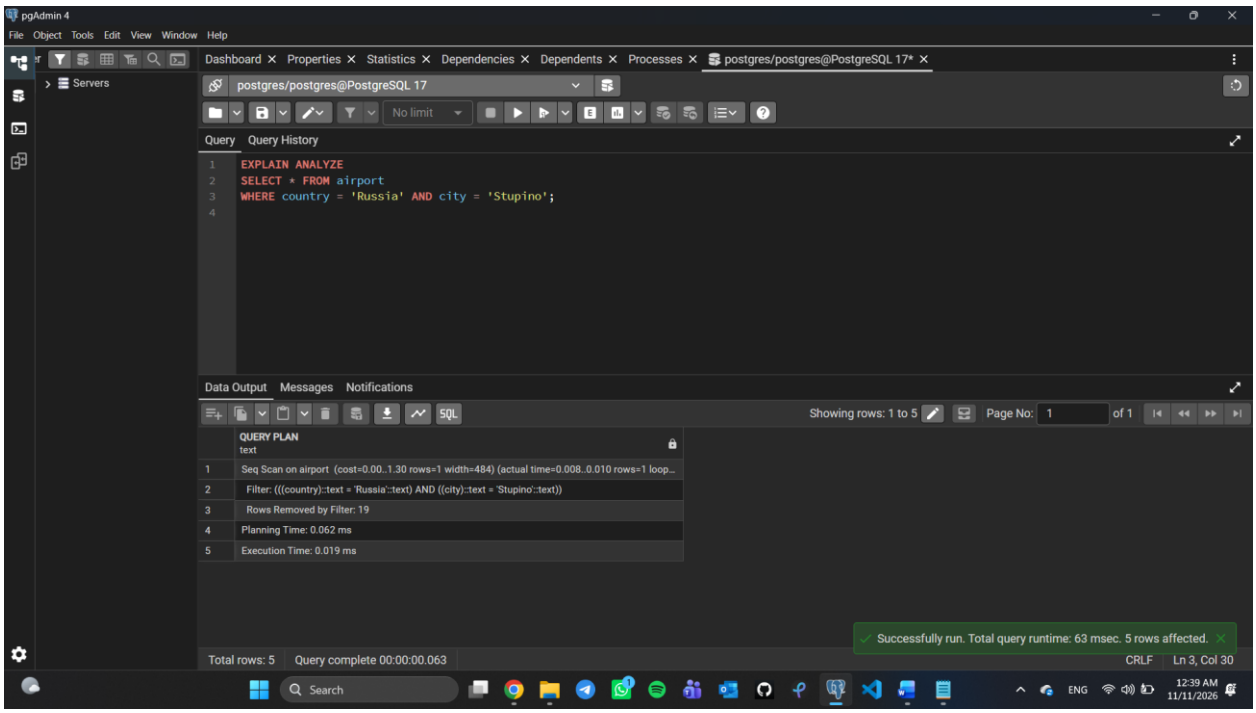
13. Create an index on the airport_name column of the airport table. Check with EXPLAIN ANALYZE whether it is used by the queries LIKE 'A%' and LIKE '%Airport'. Explain why the results are different.



I created an index on airport_name and checked both queries. Both of them used a Seq Scan, so the index was not used in either case. For '%Airport' this is expected, because the pattern starts with % and a B-tree index cannot search by the end of a string. For 'A%' the index could help in theory, but the airport table has only 20 rows. Reading the whole table is cheaper for the optimizer than going through the index.

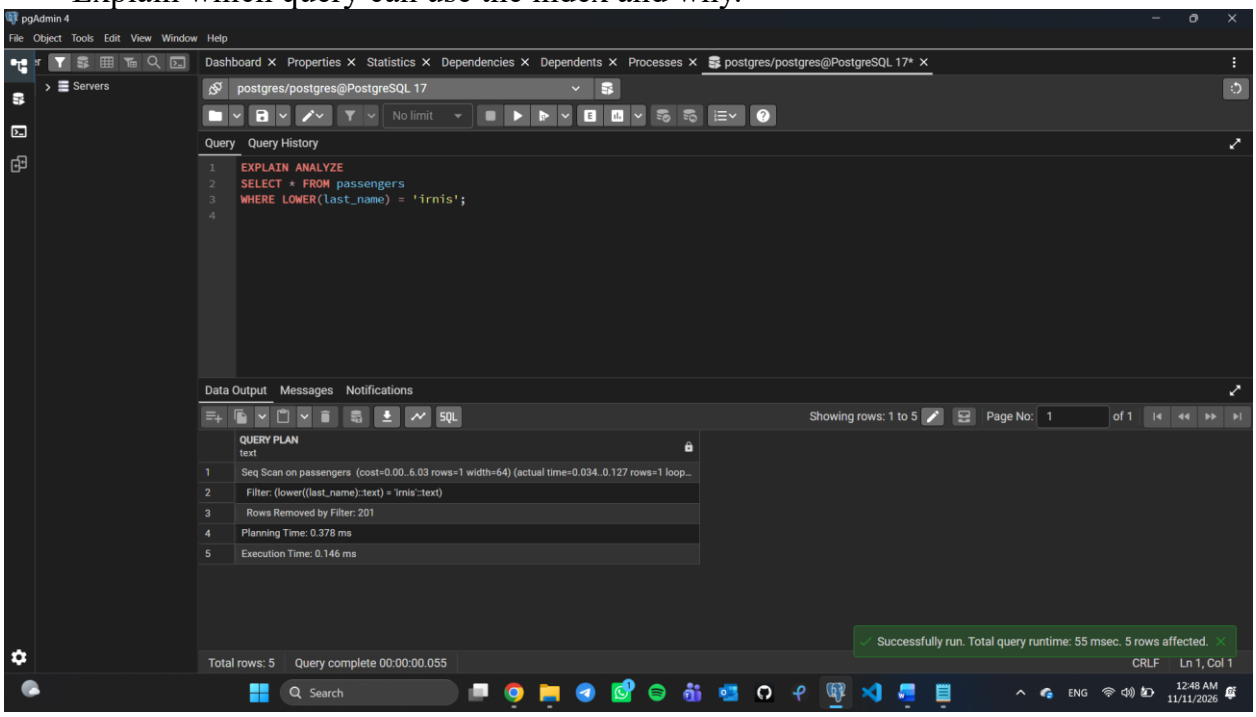
14. Create a composite index on the country and city columns of the airport table. Run three queries: filtering by country only, by city only, and by both columns. Check which of them use the index and explain the leftmost prefix rule using your results.

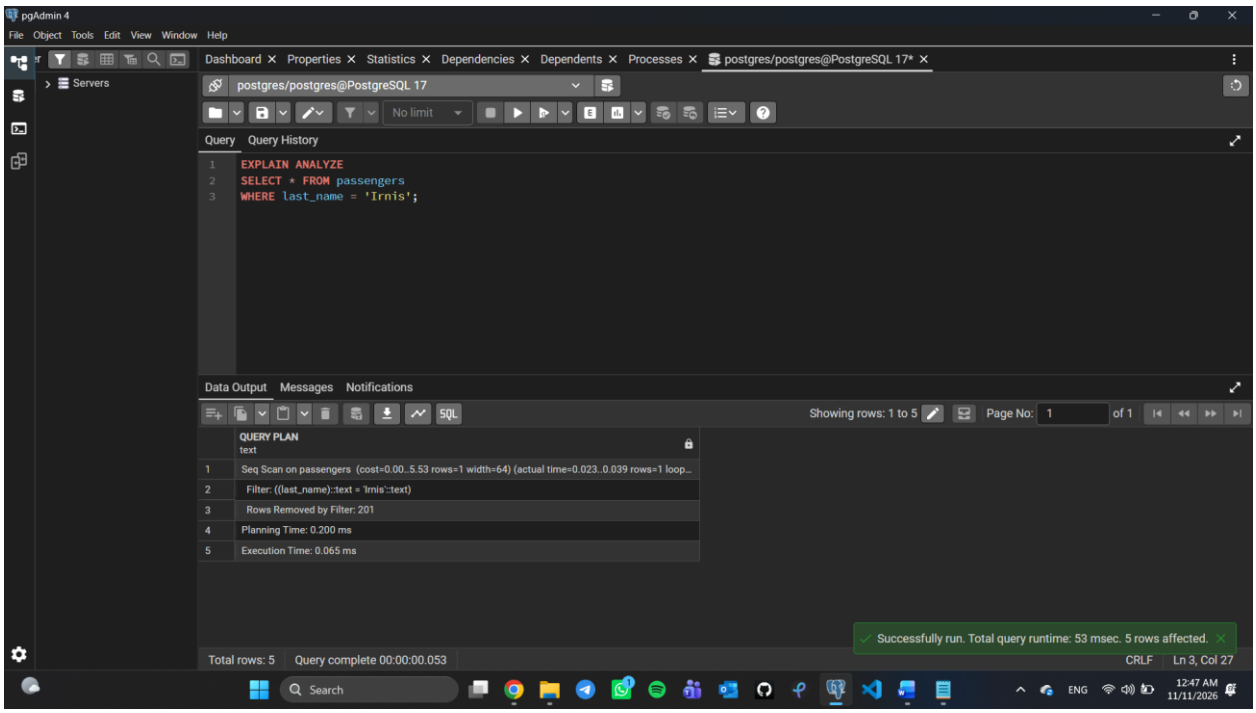




All three queries used a Seq Scan. But, normally the last query with both country and city would run faster comparing to city alone. Because, there can be multiple citieswith same name, but in AND we can jump straight to index.

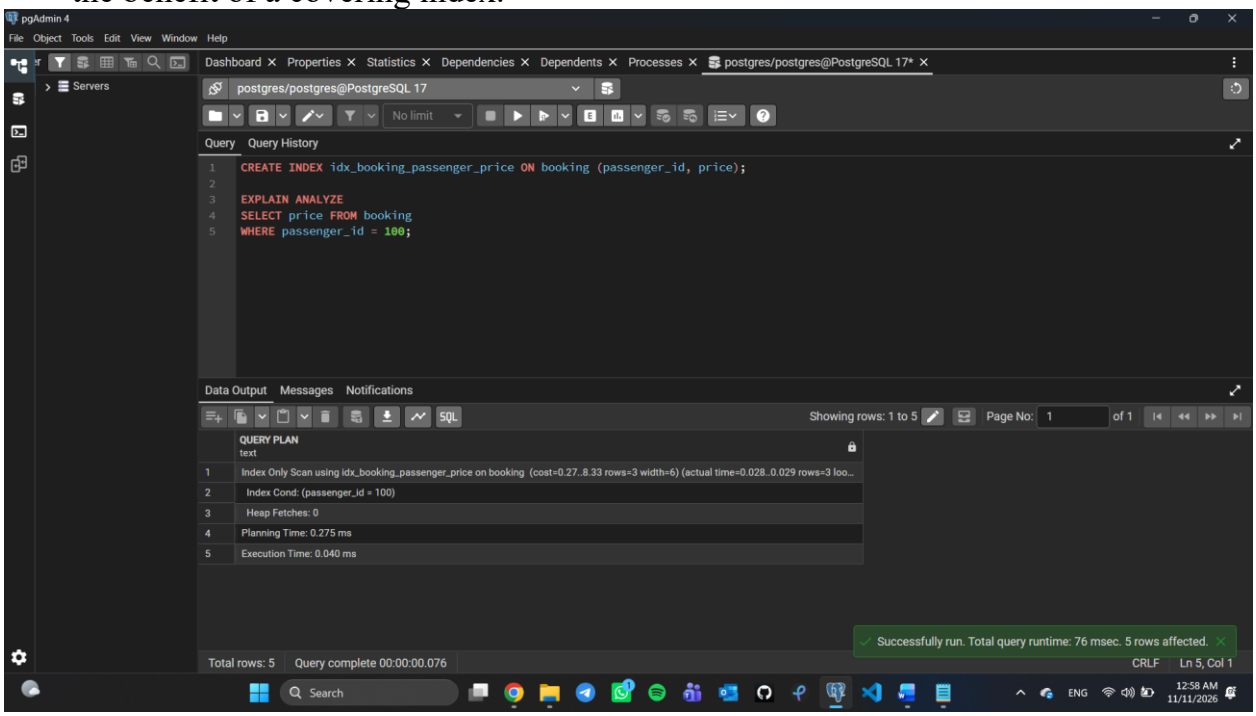
15. Create an expression (functional) index on the lower-case last_name of the Passengers table (LOWER(last_name)). Compare the plans of a query written as WHERE LOWER(last_name) = 'irnis' and a query written as WHERE last_name = 'Irnis'. Explain which query can use the index and why.





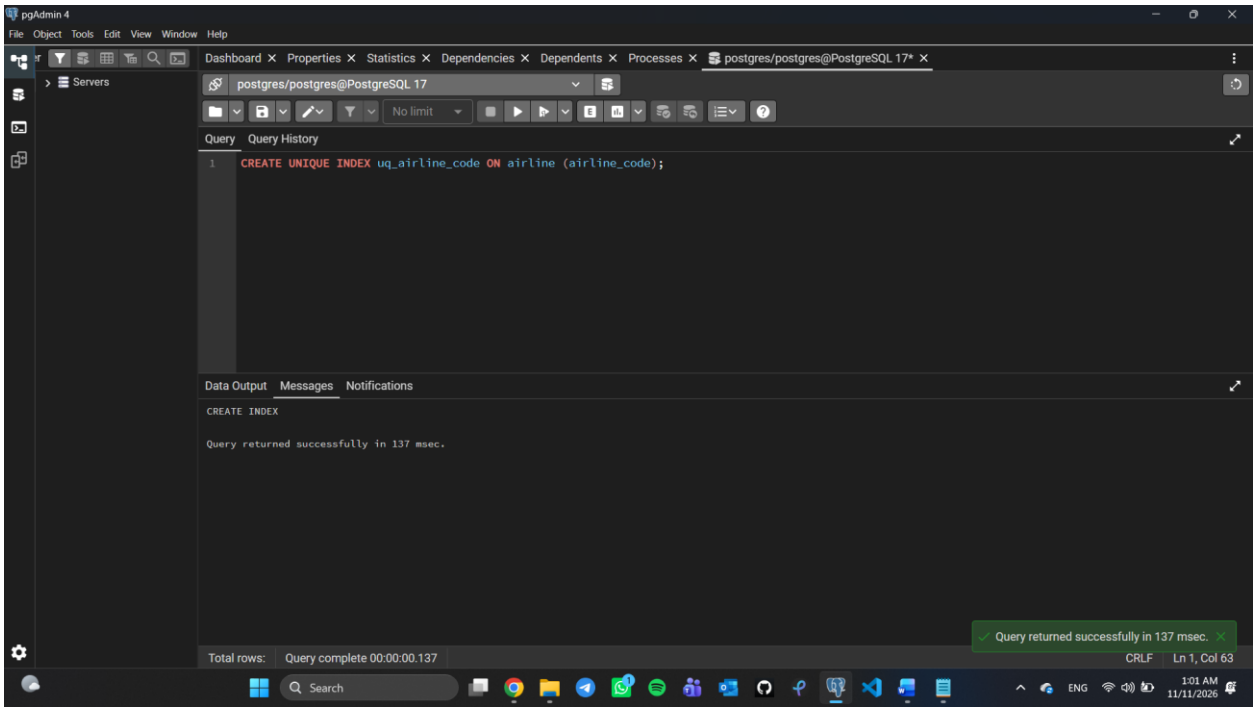
Optimizer used Seq Scan for both queries because the table is small. But still the expression index is useful for case-insensitive search and works faster within first case in larger tables.

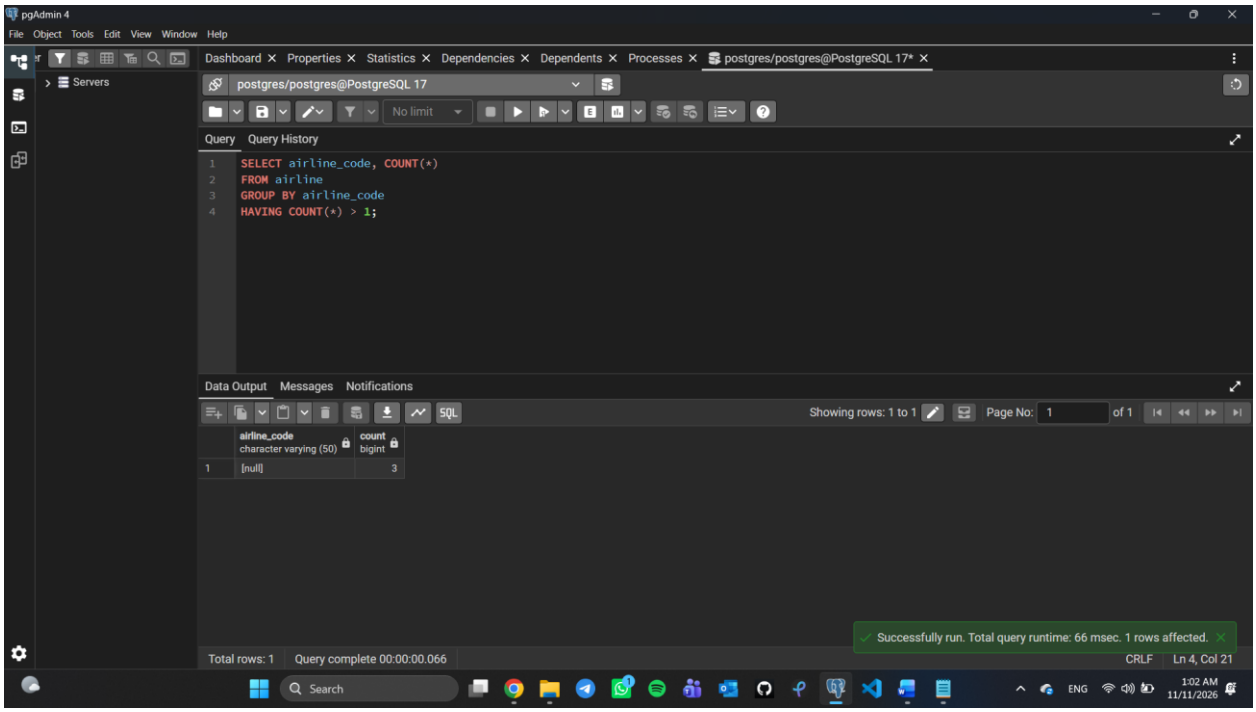
16. Create a covering index on the booking table for the columns passenger_id and price. Write a query that returns only price for a given passenger_id and check with EXPLAIN ANALYZE whether an index-only scan (or "Using index") is used. Explain the benefit of a covering index.



The benefit of a covering index is that it holds every column the query needs passenger_id for the search and price for the result. Skipping the table reads saves disk access, which matters more on large tables. The costs are a bigger index and slower inserts and updates.

17. Try to create a unique index on the airline_code column of the airline table. If an error occurs, explain what caused it and find the problematic rows with a GROUP BY ... HAVING COUNT(*) > 1 query. Then create the unique index in a way that works for the current data, or after fixing the data.





An error would only appear if two airlines had the same non-NULL code, and the test insert with an existing code SCIP shows this the database rejects it. If such duplicates existed, I would first find them with the query above, then fix or delete them, and only then create the index. But if index needs to be added without changes in data we can do: partial unique index, unique index on a combination of columns